ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR

Membre de HONORIS UNITED UNIVERSITIES

Rapport de Projet – Backoffice de Gestion « MyManager »

Module : Développement Web

Décembre 28, 2025

MiniProjet –

Backoffice de Gestion « MyManager »

REALISE PAR :

- **Mohamed Chafik El Ouazzani**
- **Saad Chafik**

Sommaire

1. Contexte et objectif du projet

2. Périmètre fonctionnel (cahier des charges)

3. Type de produits manipulés

4. Données et usage du JSON

5. Architecture technique

6. Parcours utilisateur

7. Dashboard (statistiques & graphiques)

8. Internationalisation (i18n)

9. Limites et améliorations (roadmap)

10. Conclusion

1. Contexte et objectif du projet

Dans le cadre du mini-projet « Application de Gestion Backoffice », l'objectif est de réaliser une application web de type backoffice en JavaScript (VanillaJS), HTML5 et CSS3, sans framework (React/Angular). L'application doit permettre la gestion d'au moins cinq entités via des fonctionnalités CRUD (Créer, Lire, Mettre à jour, Supprimer) et proposer un tableau de bord (dashboard) avec des indicateurs et des graphiques.

Le projet met l'accent sur la manipulation du DOM, les fonctionnalités asynchrones (fetch / promesses), l'organisation du code et l'internationalisation (i18n).

2. Périmètre fonctionnel (cahier des charges)

Le périmètre de l'application « MyManager » couvre cinq entités principales :

- Utilisateurs (Users)
- Produits (Products)
- Commandes (Orders)
- Clients (Customers)
- Factures (Invoices)

Pour chaque entité, l'application propose :

- Une page de listing sous forme de tableau (affichage paginé)
- Recherche (filtre texte)
- Tri des colonnes (ascendant/descendant)
- Actions : voir détails, ajouter, modifier, supprimer (avec confirmation)

• Export CSV (sur la liste filtrée ou complète)

Le dashboard doit inclure :

• Des cartes KPI (nombre total d'éléments, revenu total, etc.)

• Des graphiques dynamiques (Chart.js)

• Des filtres/segments (ex. statuts de commandes) – base fonctionnelle à étendre

Exigences UI du cahier des charges :

• Logo/nom de l'entreprise (branding)

• Bouton de déconnexion

• Dropdown d'internationalisation (Arabe, Français, Anglais)

3. Type de produits manipulés

Les « produits » de MyManager sont des objets de démonstration inspirés d'un catalogue e-commerce. Dans la version actuelle, les produits sont récupérés via une API publique (FakeStoreAPI). On retrouve typiquement :

• un identifiant (id)

• un titre (title)

• un prix (price)

• une catégorie (category) – ex. electronics, jewelery, men’s clothing, women’s clothing

• une image (image) fournie par l’API (URL)

Ce choix permet de montrer un backoffice réaliste : listing, ajout, modification, et affichage d’images produits.

4. Données et usage du JSON

Oui, le JSON est utilisé au cœur du projet.

1) Données chargées depuis des APIs publiques (format JSON)

• Users : JSONPlaceholder (https://jsonplaceholder.typicode.com/users)

• Products : FakeStoreAPI (https://fakestoreapi.com/products)

Les réponses fetch() sont des objets JSON, stockés ensuite dans app.data.

2) Données simulées (mock)

Lorsque l’API n’est pas disponible, l’application génère des données fictives (mock) sous forme d’objets JavaScript. Ces objets suivent une structure équivalente à des fichiers JSON (listes d’objets).

3) Stockage (amélioration recommandée)

Pour un projet 100% front, il est recommandé de persister les ajouts/modifications via localStorage (sérialisation JSON), afin de ne pas perdre les données au rafraîchissement.

5. Architecture technique

Le projet est structuré comme une application « Single Page » (SPA légère) :

• Une page HTML principale

• Des sections/pages (div.page) affichées/masquées via JavaScript

• Un objet central « app » qui encapsule :

- les données (app.data)

- l'état (page courante, langue, pagination, tri, filtres)

- les méthodes (init, loadData, renderTable, createCharts, etc.)

Principales bibliothèques tierces :

• Chart.js : graphiques du dashboard

• jsPDF : export PDF (prévu pour les détails facture)

Technos :

• HTML5 / CSS3

• JavaScript Vanilla (ES6+), Promesses, async/await

• Manipulation du DOM via document.querySelector / addEventListener

6. Parcours utilisateur

1) Authentification

• Écran de connexion (login)

• Vérification simple (admin/admin) pour la démonstration

• Redirection vers l'application (affichage du backoffice)

2) Navigation

• Sidebar (menu) : Dashboard, Utilisateurs, Produits, Commandes, Clients, Factures

• Topbar : titre de page, sélecteur de langue, bouton de déconnexion

3) Gestion des entités

• Consultation (table)

• Recherche / tri / pagination

• Actions CRUD via fenêtre modale (add/edit/view)

• Suppression avec confirmation (confirm)

4) Exports

• CSV depuis chaque module

• PDF : prévu dans le module Factures (à finaliser via jsPDF)

7. Dashboard (statistiques & graphiques)

Le dashboard fournit un aperçu rapide de l'activité :

• Cartes KPI : total utilisateurs, produits, commandes, revenu total

• Graphiques : ligne (croissance), camembert (statuts), barre (revenu mensuel), donut (catégories), scatter (corrélation panier)

Problème constaté (agrandissement « infini »)

Sur certains écrans, un graphique (souvent le camembert/donut) peut se redimensionner en boucle si :

• le canvas est dans un conteneur dont la hauteur n'est pas stabilisée, et/ou

• Chart.js est en mode responsive avec maintainAspectRatio par défaut, et/ou

• le layout change à chaque frame (scrollbar / flex / grid qui recalculent)

Correctifs recommandés :

• Donner une hauteur fixe/minimale au conteneur des charts

• Forcer Chart.js : responsive: true, maintainAspectRatio: false

• Fixer la taille du canvas via CSS (height: 280px par ex.)

• Détruire proprement les instances Chart avant recréation (déjà fait)

• Éviter de recréer les charts inutilement (ex. à chaque navigation)

8. Internationalisation (i18n)

Le projet intègre un mécanisme i18n basé sur des dictionnaires (app.translations) et des attributs data-i18n dans le HTML.

État actuel :

• Français (fr) et Anglais (en) sont implémentés.

À compléter selon le cahier des charges :

• Ajouter l'Arabe (ar) :

- nouvelles traductions dans app.translations.ar

- gestion du sens d'écriture RTL (dir="rtl") et styles CSS adaptés (alignement, sidebar, etc.)

Bonnes pratiques i18n :

• Ne pas traduire « en dur » dans le HTML : tout passer par data-i18n

• Traduire aussi les placeholders des champs (recherche)

• Prévoir les labels de formulaires / modales / messages d'alerte

9. Limites et améliorations (roadmap)

Améliorations prioritaires

1) Produits avec images (fonction "Add Product")

• Ajouter un champ « image URL » ou un upload de fichier

• Stocker l'image dans l'objet produit (image)

• Afficher une miniature (thumbnail) dans le tableau Produits

2) Dashboard plus "pro"

• Ajouter un filtre global (ex. période : 7 jours / 30 jours / 12 mois)

• Ajouter des segments cliquables (statuts commandes)

• Ajouter une table "Dernières commandes" et "Top produits"

3) Persistance locale

• Sauvegarder app.data dans localStorage après chaque CRUD

• Recharger depuis localStorage au démarrage (fallback si vide)

4) Permissions (option avancée du sujet)

• Rôles : super_admin, admin, agent, user

• Masquer/afficher des menus et actions selon le rôle (RBAC)

• Exemple : agent = lecture seule sur factures; admin = CRUD; super_admin = gestion des rôles

5) Export PDF

• Terminer l'export facture : génération d'un PDF propre (en-tête, lignes, TVA, total, date, client) via jsPDF

10. Conclusion

Le projet « MyManager » répond à l'objectif principal : construire un backoffice fonctionnel en VanillaJS avec plusieurs entités CRUD, un dashboard avec statistiques et graphiques, et un mécanisme i18n.

Les prochaines étapes pour se rapprocher pleinement du cahier des charges sont l'ajout de la langue arabe (RTL), la gestion complète des images produits (upload/URL), la stabilisation responsive du dashboard (bug Chart.js) et la persistance locale des données.

Ces améliorations rendent l’application plus réaliste, plus stable et plus conforme aux attentes d’un véritable outil de gestion.